PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Tecnólogo en Análisis y Desarrollo de Software.
- **Código del Programa de Formación**: 228118
- **Nombre del Proyecto Formativo:** Construcción de software integrador de tecnologías orientadas a servicios.
- **Fase del Proyecto:** Construcción.
- **Actividad de Proyecto Formativo:** Codificar los módulos del sistema de información.
- Competencia:
    - 220501097 - Implementar la solución de software de acuerdo con los requisitos de operación y modelos de referencia
- Resultados de Aprendizaje:
    - 220501097-04 Implantar el software de acuerdo con los niveles de servicio establecidos con el cliente.
- Duración de la Guía de Aprendizaje (horas):72 horas

2. PRESENTACIÓN

Bienvenido(a) a esta guía de aprendizaje del SENA, en la cual usted fortalecerá competencias clave para el desarrollo en C# orientadas a la implementación de soluciones reales. A través de este proceso formativo, se trabajará el Resultado de Aprendizaje **"220501097-04 Implantar el software de acuerdo con los niveles de servicio establecidos con el cliente."**, integrando fundamentos del entorno .NET y la configuración de su espacio de trabajo —incluida la instalación y creación de proyectos en Visual Studio Code— como punto de partida para construir aplicaciones con calidad, mantenibilidad y enfoque en el servicio esperado por el cliente. Durante el recorrido, usted conectará conocimientos previos de lógica de programación con la sintaxis de C#, el uso correcto de tipos de datos, operadores, control de flujo, funciones y manejo de valores nulos, para avanzar hacia el desarrollo con Programación Orientada a Objetos (clases, herencia, interfaces, excepciones y polimorfismo) y el uso eficiente de colecciones (listas, diccionarios, colas y pilas). Se promoverá el trabajo autónomo, sistemático y organizado mediante retos y ejercicios progresivos, así como el aprendizaje colaborativo para discutir decisiones de diseño y resolver problemas de forma conjunta.

Contará con el acompañamiento del instructor(a) para orientar su avance, retroalimentar sus productos y asegurar que su desempeño se alinee con las condiciones de implantación y los niveles de servicio definidos.

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

- **Descripción de la(s) Actividad(es)**

3.1 Actividades de reflexión inicial:

220501097-04-AA1-EV01 – Introducción a C# y .NET: Conceptos, evolución y áreas de aplicación.

Descripción de la actividad

Cada aprendiz elaborará una **infografía digital individual** que sintetice los conceptos clave de **C# y la plataforma .NET**, abordando: qué es C# y su relación con .NET, componentes esenciales del ecosistema (como CLR y BCL), evolución general del lenguaje y **principales áreas de aplicación** (web, escritorio, móvil, videojuegos, nube, etc.). La infografía debe presentar la información en un lenguaje técnico accesible y visualmente organizado, incluyendo **al menos un ejemplo corto de código** (snippet) y una analogía breve para facilitar la comprensión.

Instrucciones de la actividad (paso a paso)

1. **Lectura y extracción de ideas clave (individual)**
   - Lea el material "Introducción a C#".
   - Identifique y anote (en borrador) los conceptos esenciales:
     - ¿Qué es C# y cómo se relaciona con .NET?
     - Componentes mínimos del ecosistema: **CLR**, **BCL** y compilación a **IL**.
     - Evolución general del lenguaje (hitos o etapas).
     - Áreas de aplicación (ej.: web, escritorio, juegos, nube, móvil).
2. **Diseño de la infografía (entregable principal)**
   La infografía debe incluir **mínimo**:
   - **Definición breve** de C# y .NET (relación lenguaje ↔ plataforma).
   - **Componentes del ecosistema .NET** (incluya CLR y BCL, y mencione IL).
   - **Línea de tiempo resumida** (3 a 6 hitos) de la evolución de C# (sin saturar).
   - **Áreas de aplicación** (mínimo 5) con ejemplos concretos (por ejemplo: "Unity → videojuegos", "ASP.NET → web", ".NET MAUI → móvil/escritorio", "microservicios en la nube").
3. **Socialización**
   - Realice un **pitch de 3 minutos** explicando su infografía: qué incluyó, por qué lo organizó así y un ejemplo de uso real de C# (.NET) en desarrollo.

Ambiente requerido

- Ambiente de formación presencial (o remoto sincrónico) con **acceso a internet**.
- Computador con editor de texto/código y una herramienta de diseño gráfico (**Canva**, **Piktochart**, **Figma** o similar).

Estrategias o técnicas didácticas activas

- Aprendizaje basado en proyectos individuales (ABP).
- Pensamiento visual (visual thinking).
- Aprendizaje significativo (conexión teoría ↔ práctica).
- Técnica de resumen gráfico para retención y jerarquización de ideas.

Materiales de formación

- Documento guía: **"Introducción a C#"** (contenido base): https://educated-show-144.notion.site/Introducci-n-a-C-2fd4671e02a1812f9bbff6f5c66aa372
- anexo-infografia.pdf (Disponible en LMS).

Material de apoyo

- Documentación oficial de C#: (Microsoft Learn)
- Concepto del CLR (.NET): (Microsoft Learn)
- Bibliotecas base / BCL (.NET): (Microsoft Learn)
- Referencia sobre .NET MAUI (como ejemplo de área de aplicación móvil/escritorio): (Microsoft Learn)

Evidencias de aprendizaje

- **Infografía digital individual** que refleje comprensión de:
  - C# vs .NET (relación y propósito).
  - Componentes clave del ecosistema: CLR, BCL e IL.
  - Evolución general del lenguaje (hitos).
  - Áreas de aplicación y casos de uso.
- Socialización oral (pitch 3 min).

Instrumento de evaluación (rúbrica)

Duración estimada: 3 horas

3.2 Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje:

220501097-04-AA1 – EV02 - Instalación y configuración del entorno para C# (.NET) y creación de proyecto (Visual Studio o Visual Studio Code)

Descripción de la actividad

De manera **individual**, el aprendiz realizará la **instalación y configuración del entorno de desarrollo para C# (.NET)**, eligiendo una de estas herramientas: **Visual Studio** o **Visual Studio Code**. Al finalizar, deberá **crear y ejecutar un proyecto C#** para comprobar que el entorno quedó correctamente instalado y funcional, dejando evidencia en una **bitácora técnica**.

Orientaciones para el desarrollo

1) Consulta de material de apoyo

Revise los siguientes recursos antes de iniciar el laboratorio:

- "Creación de proyecto C#" (educated-show-144.notion.site)
- "C# en Visual Studio Code – instalación y creación de proyecto" (educated-show-144.notion.site)

2) Laboratorio (seleccione UNA ruta)

Ruta A: Visual Studio (IDE)

1. Descargar e instalar **Visual Studio**.
2. Durante la instalación, seleccionar el componente **Desarrollo de escritorio con .NET**.
3. Verificar instalación abriendo Visual Studio y confirmando que puede crear un proyecto de C#.
4. Crear un proyecto tipo **Aplicación de consola**.
5. Ejecutar el proyecto y validar salida en consola.

Ruta B: Visual Studio Code

1. Instalar **Visual Studio Code**.
2. Instalar el **SDK de .NET** correspondiente.
3. Instalar la extensión de C# requerida en VS Code (según guía del material).
4. Verificar instalación desde terminal:
   - `dotnet --version`
   - `dotnet --list-sdks`
5. Crear un proyecto C# tipo consola con:

- `dotnet new console`

6. Ejecutar el proyecto con:
   - `dotnet run`

Requisito técnico (común para ambas rutas)

- Instalar (y evidenciar) al menos estas versiones del SDK:
  - **.NET versión actual pero LTS (vigente)**

Evidencias de aprendizaje

Informe escrito tipo Bitácora

Debe incluir:

1. Herramienta elegida: **Visual Studio** o **Visual Studio Code**.
2. Paso a paso de instalación y configuración (con capturas).
3. Evidencias de versiones instaladas:
   - Captura del instalador o salida de `dotnet --list-sdks` (si aplica).
4. Evidencia del proyecto creado:
   - Captura de la estructura del proyecto.
5. Evidencia de ejecución exitosa:
   - Captura del programa corriendo (salida en consola).

Ambiente requerido

- Equipos de cómputo.
- Conexión a Internet.
- Permisos de instalación (si aplica).

Estrategias o técnicas didácticas activas

- **Laboratorio práctico guiado**
- **Informe tipo bitácora**

Material de apoyo

- "Creación de proyecto C#" (educated-show-144.notion.site)
- "C# en Visual Studio Code – instalación y creación de proyecto" (educated-show-144.notion.site)

Duración de la actividad: 3 horas

220501097-04-AA1 – EV03 – Aplicación de la sintaxis básica de C# - Tipos de datos, Variables, constantes, operadores e Interpolacion de String

Descripción de la actividad

En esta actividad, el aprendiz pondrá en práctica los fundamentos de la **sintaxis básica de C#**, desarrollando pequeños programas que integren:

- Declaración y uso de **tipos de datos**
- Definición de **variables y constantes**
- Uso de **operadores aritméticos y de asignación**
- Construcción de salidas por consola usando **interpolación de strings**

La actividad se apoya directamente en los materiales formativos suministrados y culmina con la resolución de **retos prácticos**, fortaleciendo la lógica de programación y la correcta escritura de código.

Materiales de formación

Los siguientes recursos deben ser estudiados y utilizados como base para el desarrollo de la actividad:

- Lección **Tipos de datos**
- Lección **Variables y constantes**
- Lección **Operadores**
- Lección **Interpolación de Strings**
- **Reto tipos de datos**
- **Reto operadores**

Los materiales necesarios para la formación los encuentras en este link: https://educated-show-144.notion.site/Guia-didactica-Curso-C-2fb4671e02a18048829ec8f2c1613ace

Ambiente requerido

- Computador con sistema operativo Windows, Linux o macOS
- Editor de código **Visual Studio Code**
- SDK de **.NET** instalado
- Consola / terminal de comandos

Estrategia didáctica

- Aprendizaje basado en la práctica
- Resolución de retos guiados
- Programación individual con retroalimentación del instructor

Instrucciones para el aprendiz

1. Estudie cuidadosamente los materiales formativos sobre:
    - Tipos de datos
    - Variables y constantes
    - Operadores
    - Interpolación de strings
2. Cree una solución de trabajo llamada:
3. `Sintaxis_CSharp`
4. Dentro de la solucion, cree los proyectos para cada ejercicio.
5. Desarrolle los siguientes ejercicios:
    - **Ejercicio 1:** Declaración e impresión de variables con distintos tipos de datos.
    - **Ejercicio 2:** Uso de operadores aritméticos y de asignación para resolver cálculos básicos.
    - **Ejercicio 3:** Mostrar resultados por consola usando **interpolación de strings**.
6. Resuelva completamente los siguientes retos, para ellos cada uno debe ser un proyecto dentro de la solucion:
    - Reto tipos de datos
    - Reto operadores
7. Verifique que el código compile y se ejecute correctamente desde consola.

Evidencias de aprendizaje

- Enlace de github con solución y los proyectos con el código fuente funcional.
- Ejecución correcta de los programas en consola.
- Resolución completa de los retos planteados.

Duración estimada: 6 horas

3.3 Actividades de apropiación:

3.4 Actividades de Transferencia el Conocimiento:

4. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN EN EL PROCESO FORMATIVO.

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
		220501097-04-AA1-EV01 – Introducción a C# y .NET: Conceptos, evolución y áreas de aplicación.	**Infografía digital individual**		Observación – Lista de Chequeo.
		220501097-04-AA1 – EV02 - Instalación y configuración del entorno para C# (.NET) y creación de proyecto (Visual Studio o Visual Studio Code)	**Informe escrito tipo Bitácora**		Observación – Lista de Chequeo.
		220501097-04-AA1 – EV03 – Aplicación de la sintaxis básica de C# - Tipos de datos, Variables, constantes, operadores e Interpolacion de String	**Repositorio de código con la solución de solicitada.**		Valoracion de producto – Lista de Chequeo.

5. GLOSARIO DE TÉRMINOS

- **C#**: Lenguaje de programación moderno, orientado a objetos y desarrollado por Microsoft. Es parte del ecosistema .NET.
- **.NET**: Plataforma de desarrollo de Microsoft para construir aplicaciones multiplataforma, incluyendo web, móviles y de escritorio.



- **Variable**: Espacio en memoria que almacena un valor que puede cambiar durante la ejecución del programa.
- **Constante**: Valor fijo que no puede modificarse después de su declaración.
- **Tipo de dato**: Categoría que define el tipo de valores que puede almacenar una variable, como `int`, `string`, `bool`, entre otros.
- **Operador**: Símbolo que realiza operaciones sobre variables y valores (ej. `+`, `-`, `==`).
- **Interpolación de Strings**: Técnica para incrustar expresiones dentro de cadenas de texto usando `$"Texto {variable}"`.
- **Condicional**: Instrucción que permite ejecutar código dependiendo del cumplimiento de una condición (`if`, `else`).
- **Estructura iterativa**: Permite repetir un bloque de código varias veces (`for`, `while`).
- **Clase**: Plantilla que define las propiedades y métodos comunes para un conjunto de objetos.
- **Objeto**: Instancia concreta de una clase.
- **Propiedad**: Atributo de una clase que permite acceder o modificar datos internos.
- **Encapsulación**: Principio que restringe el acceso directo a los datos internos de un objeto.
- **Constructor**: Método especial que se ejecuta al crear un objeto y lo inicializa.
- **Destructor**: Método que se invoca automáticamente al destruir un objeto.
- **Herencia**: Mecanismo por el cual una clase puede heredar atributos y métodos de otra clase.
- **Polimorfismo**: Capacidad de redefinir métodos en clases derivadas para modificar su comportamiento.
- **Clase abstracta**: Clase que no se puede instanciar directamente y que puede contener métodos abstractos (sin implementación).
- **Interface**: Contrato que define un conjunto de métodos que una clase debe implementar.
- **Método virtual**: Método que puede ser sobreescrito por una clase derivada usando `override`.
- **Sobrecarga**: Definición de múltiples métodos con el mismo nombre pero diferentes parámetros.
- **Método de extensión**: Método que “extiende” una clase sin modificar su definición original.
- **Miembros estáticos**: Atributos o métodos accesibles sin crear una instancia de la clase.
- **Excepción**: Evento que ocurre cuando se produce un error durante la ejecución. Se maneja mediante `try`, `catch`, `finally`.
- **Delegado**: Tipo de dato que referencia métodos con una firma específica.
- **Evento**: Mecanismo que permite a una clase notificar a otras cuando ocurre una acción.
- **Array**: Colección de elementos de tamaño fijo y tipo homogéneo.
- **Lista (List)**: Colección dinámica que puede crecer y reducirse en tiempo de ejecución.
- **Diccionario**: Estructura de datos que almacena pares clave-valor.
- **Conjunto (HashSet)**: Colección que almacena elementos únicos, sin orden específico.
- **Pila (Stack)**: Estructura de datos LIFO (último en entrar, primero en salir).
- **Cola (Queue)**: Estructura de datos FIFO (primero en entrar, primero en salir).
- **Genérico (Generic)**: Tipo de clase o método que trabaja con cualquier tipo de dato sin perder tipado.
- **Tupla**: Colección ordenada de elementos de diferentes tipos, accesibles por posición.
- **Tipo anónimo**: Objeto sin nombre explícito de clase, útil para almacenar datos temporales.

- **Null**: Valor que representa la ausencia de datos. Puede causar errores si no se maneja correctamente.
- **Manejo de valores nulos**: Técnicas como el operador `??` o `?.` que permiten trabajar con valores que podrían ser nulos sin generar excepciones.

6. REFERENTES BILBIOGRÁFICOS

- Albahari, J., & Albahari, B. (2021). *C# 9.0 in a Nutshell: The Definitive Reference* (8th ed.). O'Reilly Media.
- Freeman, A., & Sharp, M. (2021). *Pro C# 9 with .NET 5: Foundational Principles and Practices in Programming* (10th ed.). Apress.
- Liberty, J. (2008). *Programming C# 3.0* (5th ed.). O'Reilly Media.
- Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6* (11th ed.). Apress.
- Sells, C., & Griffiths, I. (2008). *Programming Windows Presentation Foundation* (2nd ed.). O'Reilly Media.
- Microsoft Learn. (2023). *Documentación oficial de C# y .NET*. https://learn.microsoft.com/es-es/dotnet/csharp/
- Skeet, J. (2019). *C# in Depth* (4th ed.). Manning Publications.

7. CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	Carlos Enrique Navia Torres	Instructor	CTMA	29 de enero de 2026

8. CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)					